Tugas Kecil 2 IF2211 Strategi Algoritma

Semester II tahun 2025/2026

Voxelization Objek 3D menggunakan Octree

Jingglang Galih Rinenggan
(13524095)

1. Algoritma

Algoritma secara kasar bekerja dengan subdivisi sebuah octree yang diletakkan pada tengah objek hingga mencapai depth maximal. Subdivisi tersebut dihitung untuk setiap face yang dimiliki objek input. Pada setiap subdivisi, sebuah node octree dibagi menjadi 8, tetapi tidak semua child node mencakup objek tersebut sehingga tidak perlu dilakukan penelusuran lebih lanjut. Hal itu menerapkan algoritma Divide and Conquer dan mempercepat program.

Pertama, program perlu menentukan posisi paling tengah dari objek dengan mengambil titik tengah dari vertex paling negatif dan vertex paling positif. Untuk setiap face yang disimpan objek, lakukan pengecekan terhadap voxel yang dia berada dengan fungsi `IsFaceOverlapWithVoxel`. Penentuan voxel mana yang beririsan dilakukan dengan subdivisi octree secara rekursif hingga mencapai kedalaman rekursi sesuai max depth yang diinput. Voxel yang tidak overlap, tidak perlu dilakukan rekursif lanjut untuk voxel tersebut.

```
func (n *OctreeNode) InsertFace(face [3]Vector3f) {
   root := n.root
   if root == nil {
       root = n
   }

   n.mu.Lock()
   root.created[n.depth]++
   n.mu.Unlock()

   if !IsFaceOverlapWithVoxel(n.Position, n.size, face) {
       n.mu.Lock()
       root.skipped[n.depth]++
       n.mu.Unlock()
       return
   }

   if n.depth == n.maxDepth {
       n.mu.Lock()
       n.Occupied = true
       n.mu.Unlock()
       return
   }
```

```
    n.split()
    for _, child := range n.Children {
        child.InsertFace(face)
    }
}
```

Pada setiap subdivisi, octree dibagi menjadi 8 voxel lebih kecil di dalamnya menggunakan fungsi sebagai berikut.

```
func (n *OctreeNode) split() {
    n.splitOnce.Do(func() {
        n.mu.Lock()
        defer n.mu.Unlock()

        if !n.IsLeaf {
            return
        }

        n.IsLeaf = false
        childSize := n.size / 2

        for i := range 8 {
            center := n.childCenter(i)
            n.Children[i] = NewOctreeNode(center.X, center.Y, center.Z,
n.depth+1, n.maxDepth, childSize, n.root)
        }
    })
}
```

Algoritma yang digunakan untuk menghitung apakah suatu voxel overlap dengan face menggunakan penerapan dari algoritma berikut: [Fast 3D Triangle-Box Overlap Testing]

```
func IsFaceOverlapWithVoxel(center Vector3f, size float64, face [3]Vector3f)
bool {
    center3 := toArray3(center)
    v := [3][3]float64{}
    for i := range 3 {
        v[i] = sub3(toArray3(face[i]), center3)
    }
```

```
   e := [3][3]float64{}
   e[0] = sub3(v[1], v[0])
   e[1] = sub3(v[2], v[1])
   e[2] = sub3(v[0], v[2])

   halfSize := size / 2

   e0 := e[0]
   e1 := e[1]
   e2 := e[2]

   fex := math.Abs(e0[axX])
   fey := math.Abs(e0[axY])
   fez := math.Abs(e0[axZ])

   // Edge e0 axis tests: X01, Y02, Z12
   if !axisIntervalOverlap(axisX(e0, v[0]), axisX(e0, v[2]),
fez*halfSize+fey*halfSize) {
       return false
   }
   if !axisIntervalOverlap(axisY(e0, v[0]), axisY(e0, v[2]),
fez*halfSize+fex*halfSize) {
       return false
   }
   if !axisIntervalOverlap(axisZ(e0, v[1]), axisZ(e0, v[2]),
fey*halfSize+fex*halfSize) {
       return false
   }

   fex = math.Abs(e1[axX])
   fey = math.Abs(e1[axY])
   fez = math.Abs(e1[axZ])

   // Edge e1 axis tests: X01, Y02, Z0
   if !axisIntervalOverlap(axisX(e1, v[0]), axisX(e1, v[2]),
fez*halfSize+fey*halfSize) {
       return false
   }
   if !axisIntervalOverlap(axisY(e1, v[0]), axisY(e1, v[2]),
fez*halfSize+fex*halfSize) {
```

```
        return false
    }
    if !axisIntervalOverlap(axisZ(e1, v[0]), axisZ(e1, v[1]),
fey*halfSize+fex*halfSize) {
        return false
    }

    fex = math.Abs(e2[axX])
    fey = math.Abs(e2[axY])
    fez = math.Abs(e2[axZ])

    // Edge e2 axis tests: X2, Y1, Z12
    if !axisIntervalOverlap(axisX(e2, v[0]), axisX(e2, v[1]),
fez*halfSize+fey*halfSize) {
        return false
    }
    if !axisIntervalOverlap(axisY(e2, v[0]), axisY(e2, v[1]),
fez*halfSize+fex*halfSize) {
        return false
    }
    if !axisIntervalOverlap(axisZ(e2, v[1]), axisZ(e2, v[2]),
fey*halfSize+fex*halfSize) {
        return false
    }

    for axis := range 3 {
        minVal := v[0][axis]
        maxVal := v[0][axis]
        for i := 1; i < 3; i++ {
            if v[i][axis] < minVal {
                minVal = v[i][axis]
            }
            if v[i][axis] > maxVal {
                maxVal = v[i][axis]
            }
        }

        if minVal > halfSize || maxVal < -halfSize {
            return false
        }
    }
```

```
    normal := [3]float64{
        e0[axY]*e1[axZ] - e0[axZ]*e1[axY],
        e0[axZ]*e1[axX] - e0[axX]*e1[axZ],
        e0[axX]*e1[axY] - e0[axY]*e1[axX],
    }
    radius := halfSize * (math.Abs(normal[axX]) + math.Abs(normal[axY]) +
math.Abs(normal[axZ]))
    planeDistance := normal[axX]*v[0][axX] + normal[axY]*v[0][axY] +
normal[axZ]*v[0][axZ]
    if math.Abs(planeDistance) > radius {
        return false
    }

    return true
}
```

Setelah semua face telah diperhitungkan, voxel octree akan dikonversi menjadi obj dengan dilakukan traversal dan menggambarkan face yang tidak bertetanggaan dengan voxel lain untuk menghemat kompleksitas objek.

```
func (n *OctreeNode) writeOccupiedVoxelsObj(w *bufio.Writer, vertexOffset
*int, stats *ObjExportStats, minCorner Vector3f, step float64, occupied
map[voxelKey]struct{}) error {
    if n.IsLeaf {
        if n.Occupied {
            stats.Voxels++
            key := voxelKeyFromCenter(n.Position, minCorner, step)
            wrote, err := writeVoxelCubeObj(w, n.Position, n.size,
*vertexOffset, stats, key, occupied)
            if err != nil {
                return err
            }
            if wrote {
                *vertexOffset += 8
            }
        }
        return nil
```

```
    }

    for _, child := range n.Children {
        if child != nil {
            if err := child.writeOccupiedVoxelsObj(w, vertexOffset, stats,
minCorner, step, occupied); err != nil {
                return err
            }
        }
    }

    return nil
}
```

```
func writeVoxelCubeObj(w *bufio.Writer, center Vector3f, size float64,
vertexOffset int, stats *ObjExportStats, key voxelKey, occupied
map[voxelKey]struct{}) (bool, error) {
    half := size / 2
    cx := center.X
    cy := center.Y
    cz := center.Z

    faceDefs := []struct {
        neighbor voxelKey
        triA     [3]int
        triB     [3]int
    }{
        {neighbor: voxelKey{X: 0, Y: 0, Z: -1}, triA: [3]int{1, 3, 2}, triB:
[3]int{1, 4, 3}},
        {neighbor: voxelKey{X: 0, Y: 0, Z: 1}, triA: [3]int{5, 6, 7}, triB:
[3]int{5, 7, 8}},
        {neighbor: voxelKey{X: 0, Y: -1, Z: 0}, triA: [3]int{1, 2, 6}, triB:
[3]int{1, 6, 5}},
        {neighbor: voxelKey{X: 0, Y: 1, Z: 0}, triA: [3]int{4, 7, 3}, triB:
[3]int{4, 8, 7}},
        {neighbor: voxelKey{X: -1, Y: 0, Z: 0}, triA: [3]int{1, 8, 4}, triB:
[3]int{1, 5, 8}},
        {neighbor: voxelKey{X: 1, Y: 0, Z: 0}, triA: [3]int{2, 3, 7}, triB:
[3]int{2, 7, 6}},
    }
```

```
    visibleFaces := make([][2][3]int, 0, 6)
    for _, def := range faceDefs {
        neighbor := voxelKey{X: key.X + def.neighbor.X, Y: key.Y +
def.neighbor.Y, Z: key.Z + def.neighbor.Z}
        if _, exists := occupied[neighbor]; exists {
            continue
        }
        visibleFaces = append(visibleFaces, [2][3]int{def.triA, def.triB})
    }

    if len(visibleFaces) == 0 {
        return false, nil
    }

    verts := [8]Vector3f{
        {X: cx - half, Y: cy - half, Z: cz - half},
        {X: cx + half, Y: cy - half, Z: cz - half},
        {X: cx + half, Y: cy + half, Z: cz - half},
        {X: cx - half, Y: cy + half, Z: cz - half},
        {X: cx - half, Y: cy - half, Z: cz + half},
        {X: cx + half, Y: cy - half, Z: cz + half},
        {X: cx + half, Y: cy + half, Z: cz + half},
        {X: cx - half, Y: cy + half, Z: cz + half},
    }

    for _, v := range verts {
        if _, err := fmt.Fprintf(w, "v %.9f %.9f %.9f\n", v.X, v.Y, v.Z); err
!= nil {
            return false, err
        }
        stats.Vertices++
    }

    for _, face := range visibleFaces {
        for _, tri := range face {
            if _, err := fmt.Fprintf(w, "f %d %d %d\n", tri[0]+vertexOffset,
tri[1]+vertexOffset, tri[2]+vertexOffset); err != nil {
                return false, err
            }
            stats.Faces++
```

```
        }
    }

    return true, nil
}
```

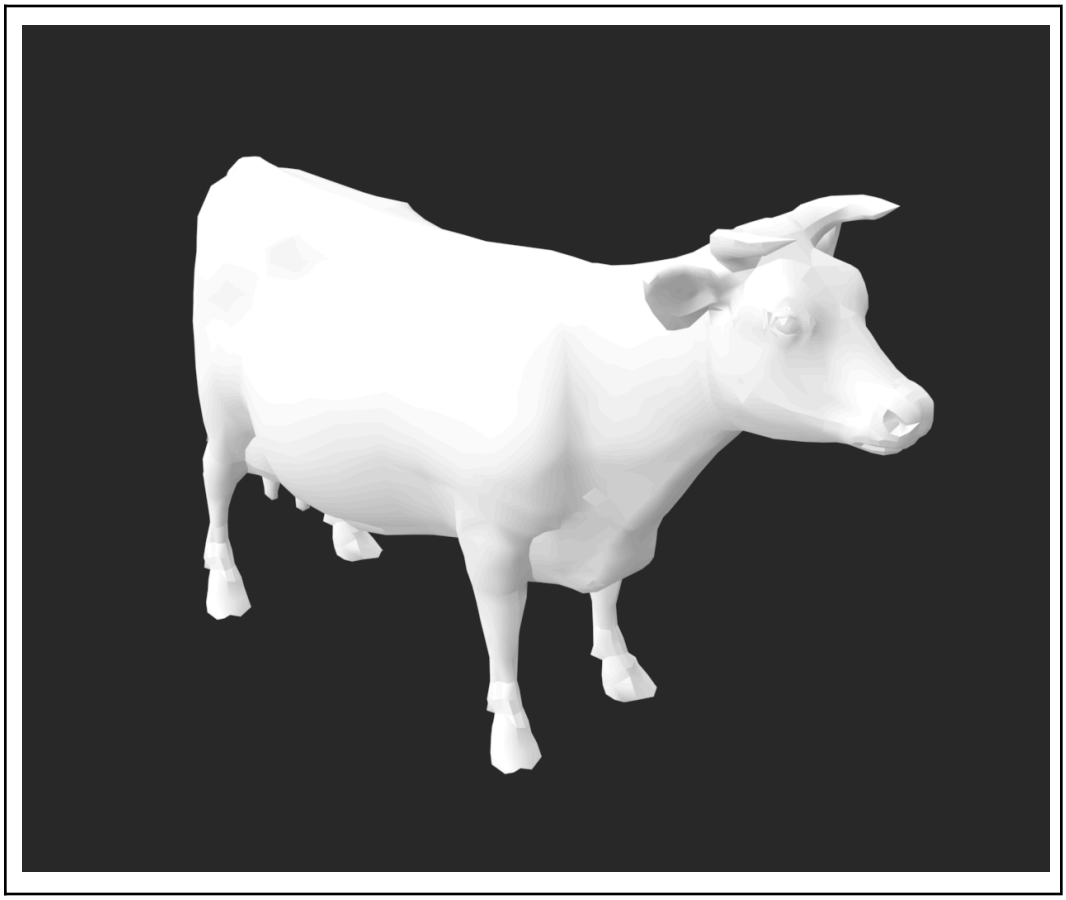
Setelah obj ditulis pada file dengan format “[Name]_res.obj”, obj tersebut ditampilkan pada GUI dengan library MeshView.

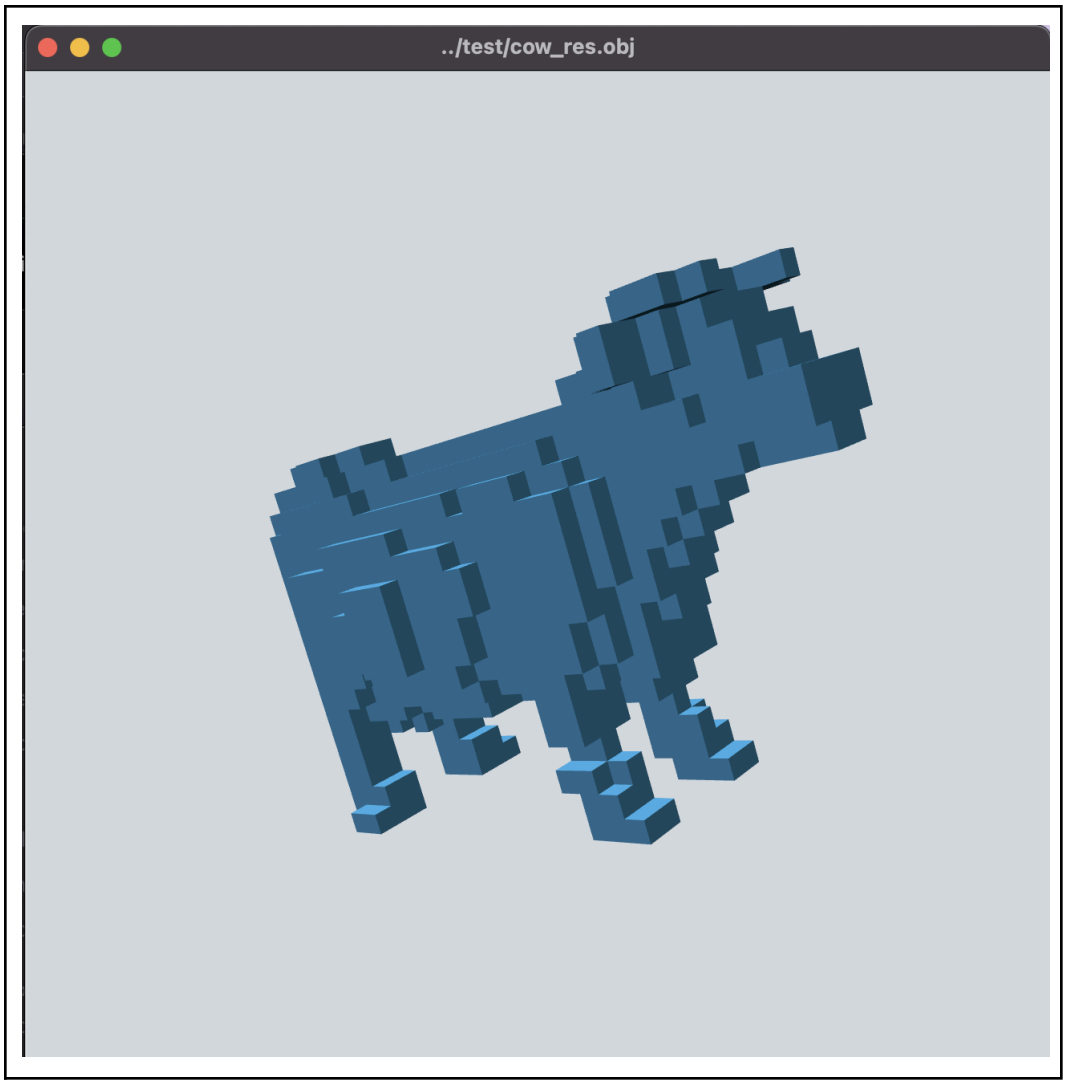
```
meshview.Run(outputPath)
```

Visualisasi ini merupakan implementasi minimal tanpa penulisan obj viewer tingkat rendah.

2. Test Case

a. Cow.obj (Depth = 5)

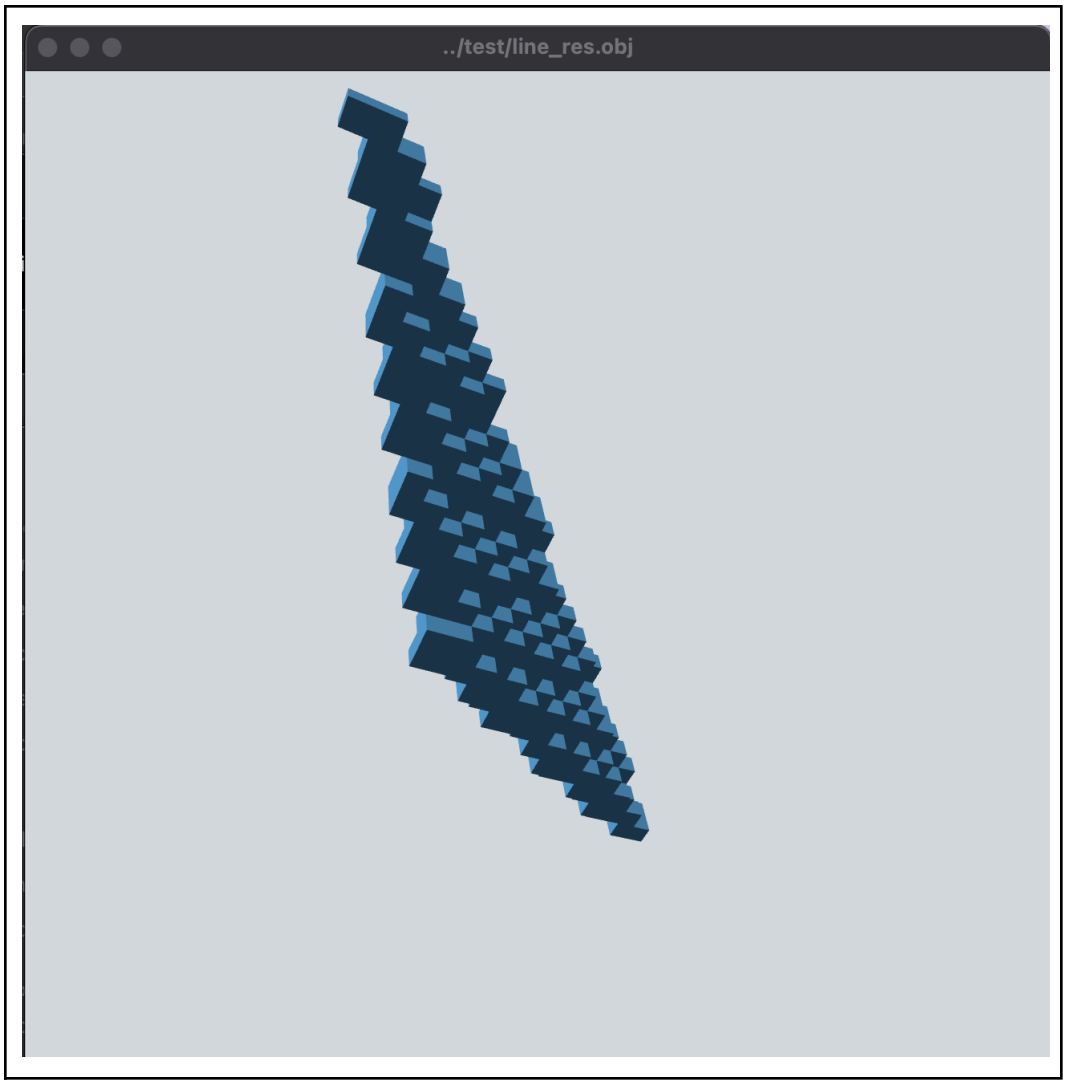



../test/cow_res.obj

```
Enter an .obj file path (without folder): cow.obj
Reading ../data/cow.obj
Enter depth: 5
Loaded Vertices: 4583
Loaded Faces: 5804
Voxels: 1362
Vertices: 10512
Faces: 5316
Node Created:
0 : 5804
1 : 44342
2 : 49353
3 : 52035
4 : 60514
5 : 73318
Node Skipped:
0 : 0
1 : 37559
2 : 42050
3 : 43630
4 : 49987
5 : 58922
Depth: 5
Result written to: ../test/cow_res.obj
Time: 42.67ms
```

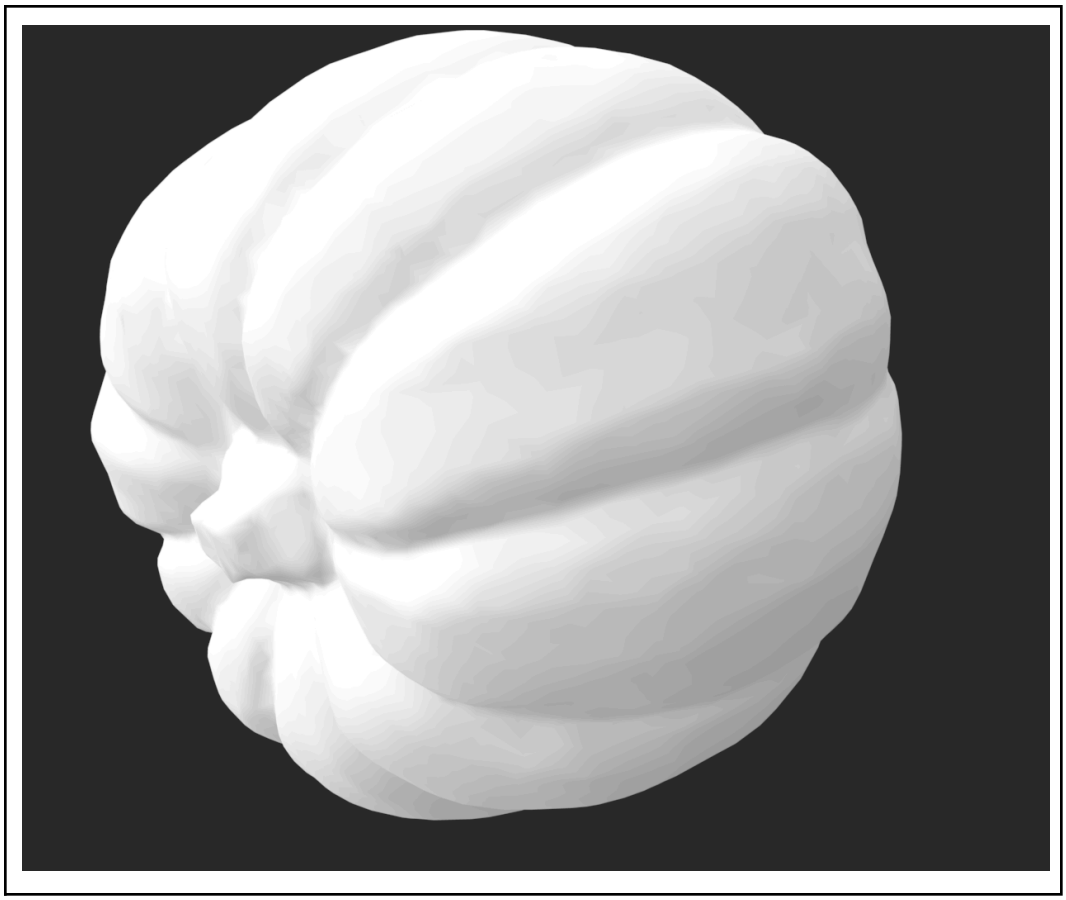
b. line.obj (Depth = 5)

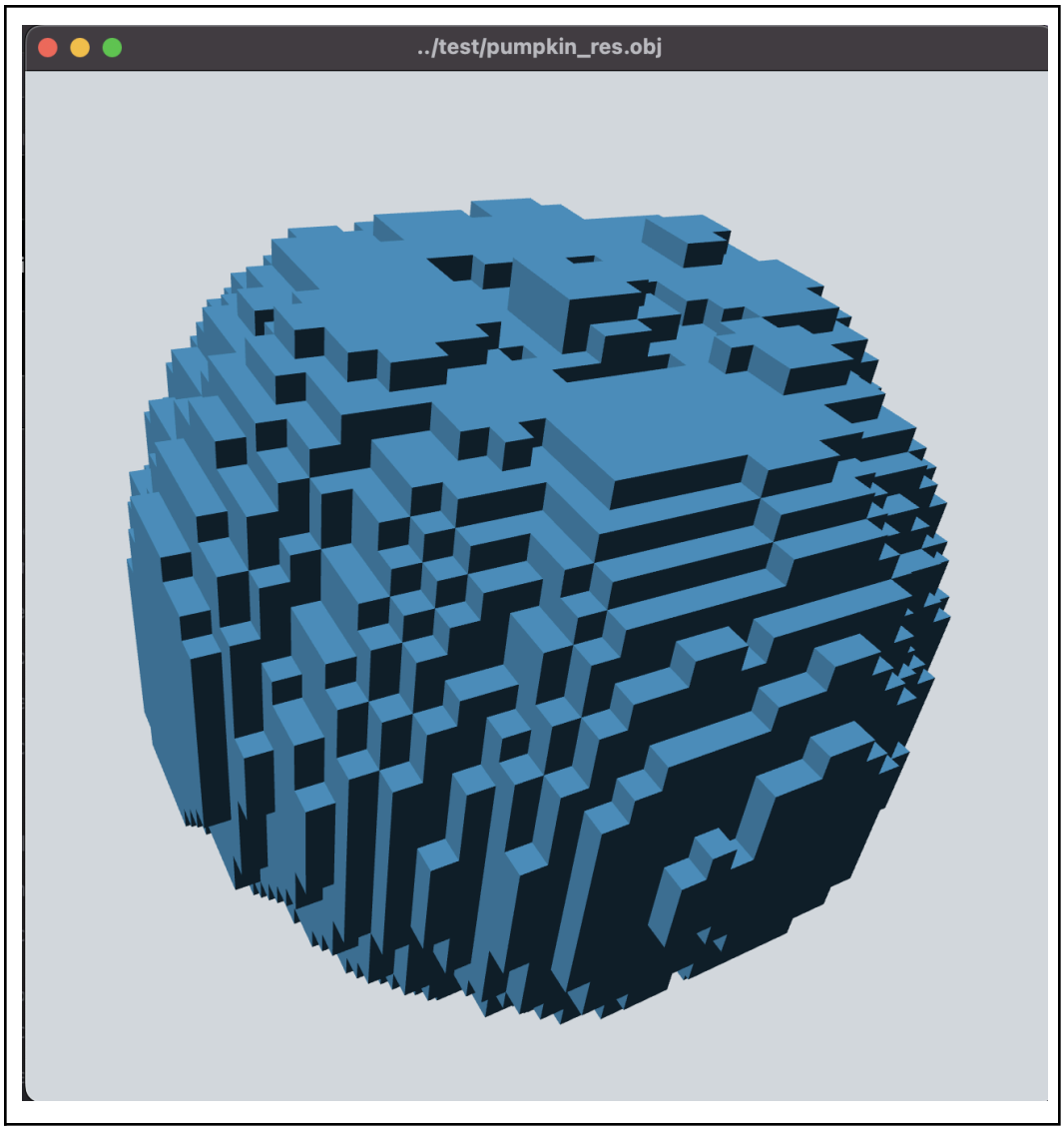
[redacted]


../test/line_res.obj

```
Enter an .obj file path (without folder): line.obj
Reading ../data/line.obj
Enter depth: 5
Loaded Vertices: 3
Loaded Faces: 1
Voxels: 287
Vertices: 2136
Faces: 1416
Node Created:
0 : 1
1 : 8
2 : 56
3 : 120
4 : 264
5 : 744
Node Skipped:
0 : 0
1 : 1
2 : 41
3 : 87
4 : 171
5 : 457
Depth: 5
Result written to: ../test/line_res.obj
Time: 13.185125ms
```

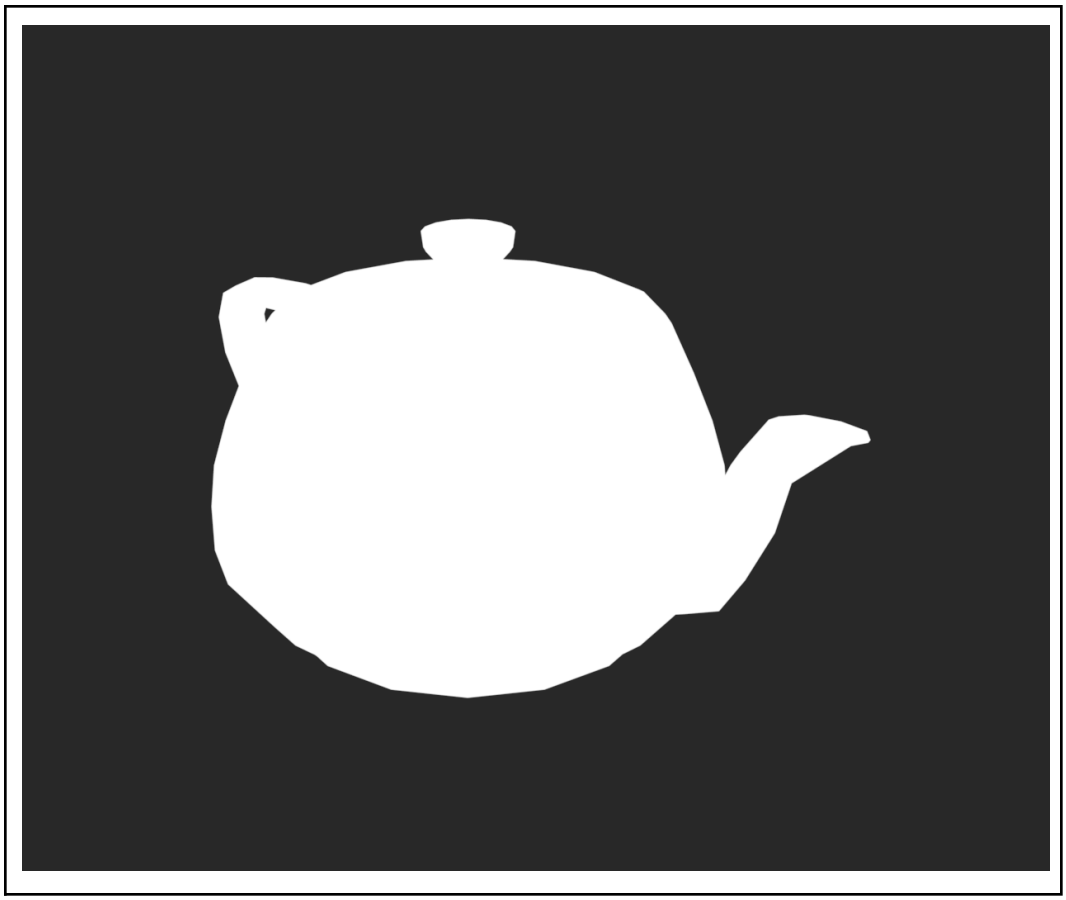
c. Pumpkin.obj (Depth = 5)

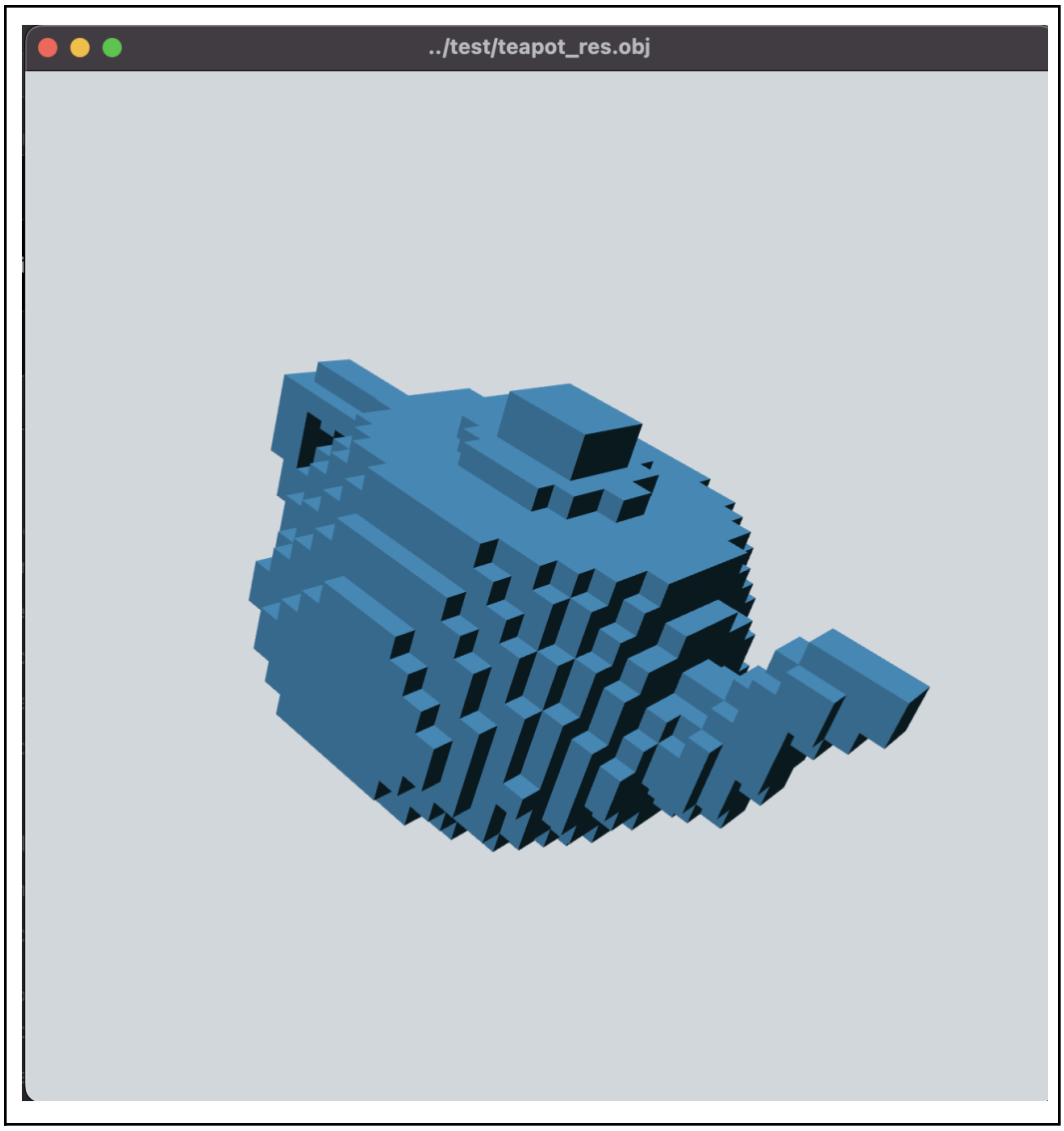



../test/pumpkin_res.obj

```
Enter an .obj file path (without folder): pumpkin.obj
Reading ../data/pumpkin.obj
Enter depth: 5
Loaded Vertices: 5002
Loaded Faces: 10000
Voxels: 4626
Vertices: 35440
Faces: 18124
Node Created:
0 : 10000
1 : 75922
2 : 82828
3 : 93209
4 : 113195
5 : 149356
Node Skipped:
0 : 0
1 : 65041
2 : 68587
3 : 75910
4 : 97468
5 : 127552
Depth: 5
Result written to: ../test/pumpkin_res.obj
Time: 101.5985ms
```

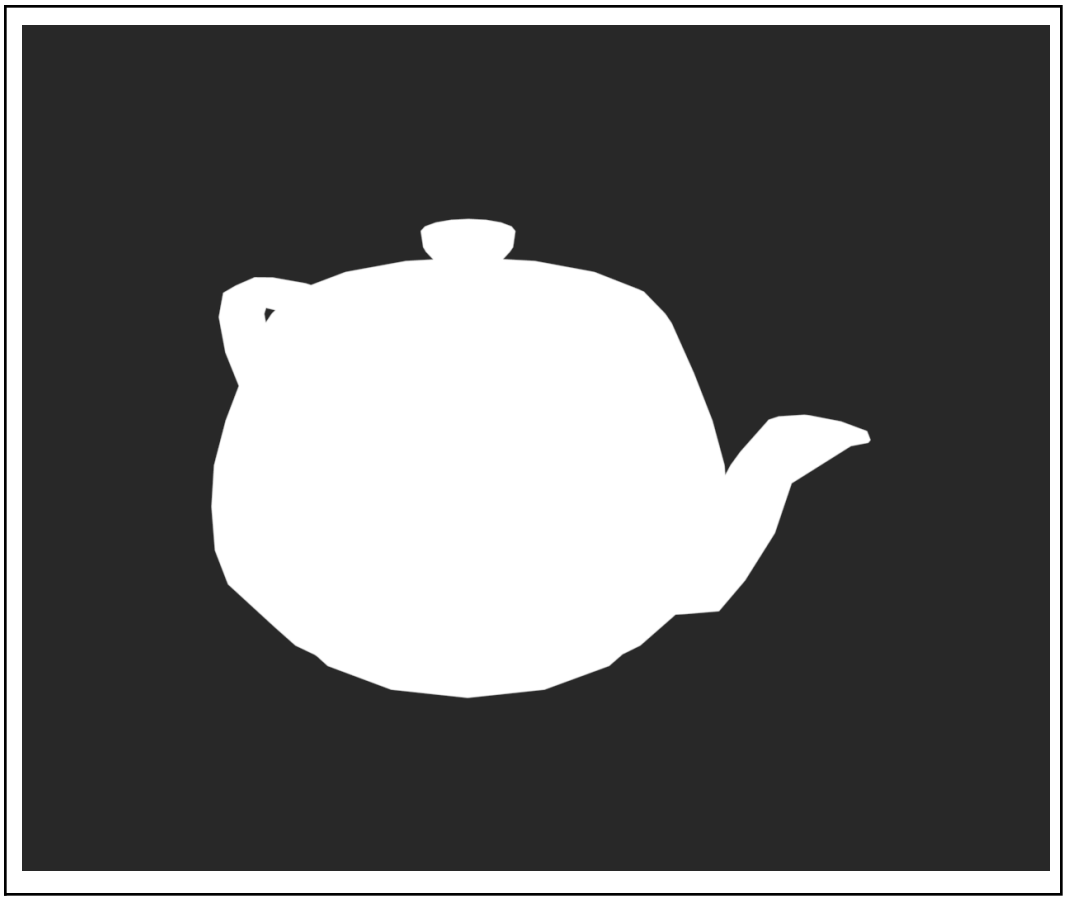
d. teapot.obj (Depth = 5)

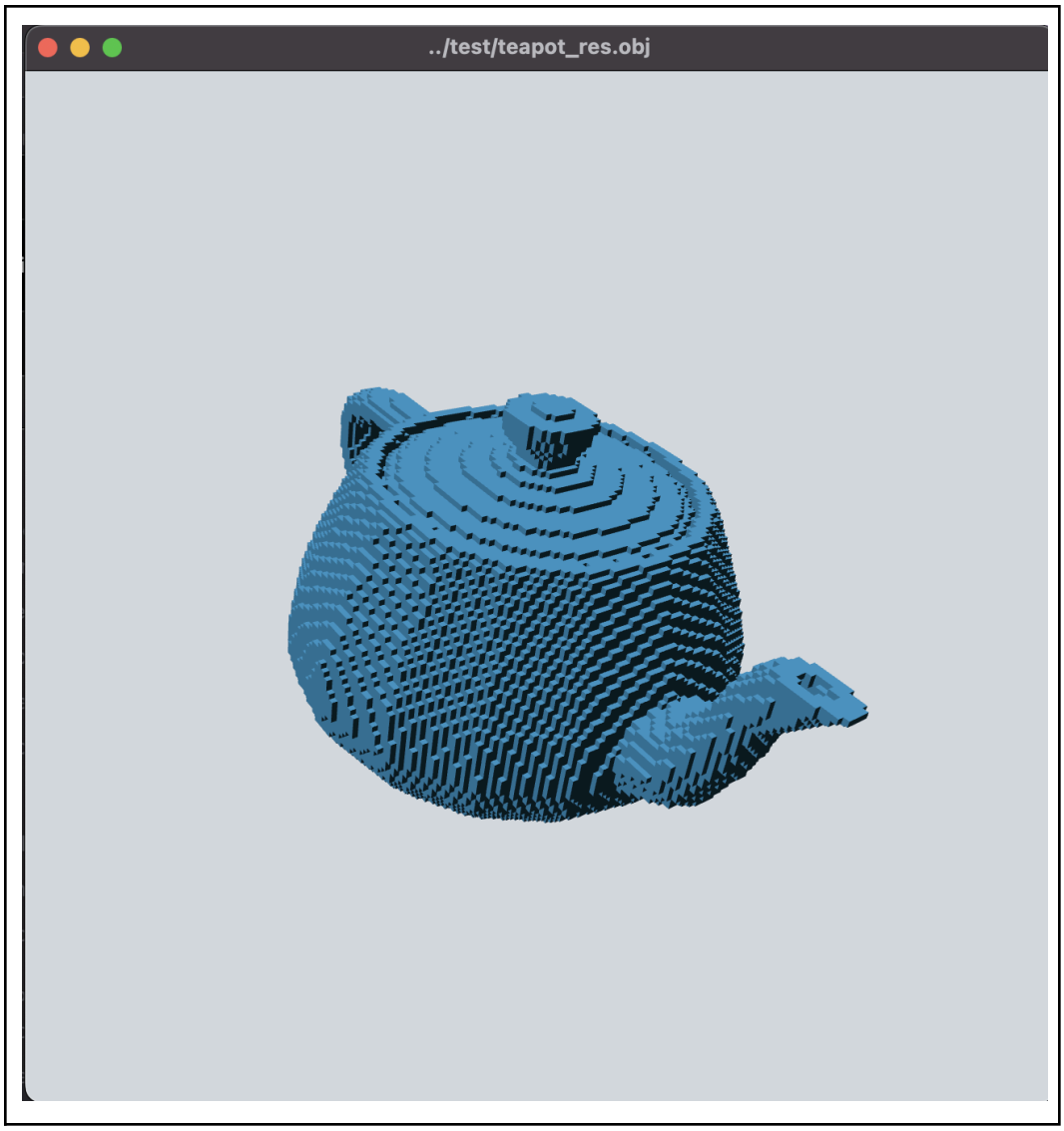



../test/teapot_res.obj

```
Enter an .obj file path (without folder): teapot.obj
Reading ../data/teapot.obj
Enter depth: 5
Loaded Vertices: 530
Loaded Faces: 1024
Voxels: 1748
Vertices: 13216
Faces: 6648
Node Created:
0 : 1024
1 : 8061
2 : 10987
3 : 12349
4 : 16636
5 : 25375
Node Skipped:
0 : 0
1 : 6666
2 : 9349
3 : 10208
4 : 13265
5 : 19376
Depth: 5
Result written to: ../test/teapot_res.obj
Time: 48.038542ms
```

e. Teapot.obj (Depth = 5)




../test/teapot_res.obj

```
Enter an .obj file path (without folder): teapot.obj
Reading ../data/teapot.obj
Enter depth: 7
Loaded Vertices: 530
Loaded Faces: 1024
Voxels: 29565
Vertices: 226464
Faces: 115660
Node Created:
0 : 1024
1 : 8060
2 : 11023
3 : 12411
4 : 16744
5 : 26180
6 : 49147
7 : 97580
Node Skipped:
0 : 0
1 : 6682
2 : 9430
3 : 10265
4 : 13380
5 : 19697
6 : 34721
7 : 69567
Depth: 7
Result written to: ../test/teapot_res.obj
Time: 328.392666ms
```

3. Lampiran

[Repository Github]

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	V	
2	Program berhasil dijalankan	V	
3	Hasil konversi .obj terdiri dari kubus-kubus kecil	V	
4	Program informasi-informasi atau report	V	

5	Program dapat menerima masukan .obj, dan mengembalikan output	V	
6	[Bonus] Program menggunakan *concurrency*	V	
7	[Bonus] Program dapat menampilkan visualisasi .obj	V	

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Jingglang Galih Rinenggan